

-



- Component Encryption

On this page

Component Encryption

Was this page helpful? 

Component encryption provides secure communication between Pulse components.

Protecting sensitive data including credit card numbers, names and addresses is a big concern in the field of risk management. To address this concern, component encryption creates a trustful environment by securing in-transit data that is communicated across Pulse components. The goal of component encryption is to ensure data privacy and integrity.

In Pulse, component encryption is implemented using SSL/TLS.

Introduction to component encryption

Component encryption aims to protect the communication between components in Pulse. This includes securing the communication between the following components:

- Internal Pulse components (e.g. between Pulse server and application engine).
- Internal and external components (e.g. between Pulse server and storage).
- External components (e.g. between various nodes of external components such as Cassandra nodes).

How does encryption work?

Pulse implements component encryption using SSL/TLS. With these protocols, the data communicated between components is encrypted using a shared secret referred to as encryption key.

The following steps summarize the process of generating and using an encryption key:

1. Server sends its public key to the client. This key is included in a digital certificate chain that contains additional information about the server.
2. Client generates the encryption key.
3. Client sends the encryption key to the server. In this step, the encryption key is encrypted with the server public key.
4. Server decrypts the encrypted key sent by the client using the server private key.
5. At this point the server and the client share the encryption key. All the communications between the server and the client during an active session can be encrypted with this key.

Authentication

In addition to encryption, SSL/TLS typically requires an authentication step to establish a connection between two components. In a typical case, the authentication is performed by the client (i.e. client authenticates the server) which is referred to as server authentication. However, authentication can be also extended to enable mutual authentication, i.e. client authenticates the server and the server authenticates the client. This latter option is commonly referred to as client authentication.

To perform server authentication, the client is required to trust a certificate presented by the server. More specifically, the component acting as a client checks:

- If the server certificate is valid.
- If the server certificate is signed by a trusted Certificate Authority (CA). There are different approaches to have signed certificates which are explained in the section [SSL/TLS certificates](#).

Component encryption with server authentication

Component encryption with server authentication requires information that must be configured on both the server and the client side to establish a secure connection:

- **Server side:** a private key and a signed certificate which certifies the information contained in the certificate.
- **Client side:** one or more certificates that the client trusts.

Typically, the keys and the certificates can be stored in various formats. In Pulse, this information is stored in Java KeyStore (JKS) format. Pulse follows a conventional nomenclature:

- **keystore:** A file that contains private keys and certificates.

A keystore file is protected with two passwords: one for the whole store and one for accessing the private key. Note that a keystore can contain more than one private key as long as each private key has a different password.

- **trustore:** A file that contains only certificates.

A certificate contains a public key and a digital signature generated with the private key of the certificate authority that issued it.

The access to a truststore file can be optionally secured with a password named truststore password. Technically, a truststore file is just keystore with no private keys; we make this distinction only to make it clear the purpose of the store.

Component encryption with server authentication only requires configuration of a keystore on the server side and a truststore on the client side.

The next section presents a diagram of the Pulse architecture for a better visualization of the components that require configuration of component encryption.

Pulse architecture with component encryption

The following diagram shows a basic Pulse architecture comprising internal Pulse components (e.g. Pulse server, application engine) and external components (e.g. Cassandra, Spark):

The numbers represent communication encryption between components. As shown in figure, the protocol SSL/TLS for component encryption is combined with additional communication protocols depending on the component.

Configuring component encryption in Pulse

To configure component encryption, Pulse requires a keystore when acting as a server and a truststore when acting as a client. The truststore has to contain a valid certificate signed by a trusted CA.

The default configuration of component encryption is done using `pulse.global` properties as described in the page [Configuration Overview](#) page.

In addition to setting default Pulse properties, there are components that cannot be configured with the default approach (e.g. relational databases) or that need custom configurations. This is described in the [Configuring Encryption for External Components](#) page.

SSL/TLS Certificates

SSL/TLS certificates are stored inside keystores and truststores which are used in the authentication process.

There are different options to configure certificates for authentication including:

- Using self-signed certificates and configuring all components to trust them.
- Using certificates signed by an internal CA and configuring all components to trust this CA.
- Using certificates signed by a commercial external CA. This is technically identical to the previous one but has an higher administrative overhead; however, well known CAs are typically trusted without requiring a truststore to be setup.

The second approach is the recommended in a production environment in Pulse, whereas the first approach is typically used in a testing environment. To help the configuration process, Pulse includes scripts to help creating keystores and truststores files with all the required certificates. Visit the [Creating Certificates](#) page for more information on how to configure these scripts.